

SILLON — Tutoriel de découverte

SQL progressif, Python et R avancés sur un jeu de données réel

À propos de ce tutoriel

Ce document accompagne le compte de démonstration `demo@sillon.local` créé par le paquet `sillon-tutoriel`. Il vous fait découvrir SILLON à travers un vrai jeu de données ouvert, avec des exercices SQL en difficulté croissante, puis un panorama complet de ce qu'il est possible de produire en Python et en R : graphiques, tableaux, exports Excel, rapports PDF et cartographie.

Identifiants du compte de démonstration

Champ	Valeur
Adresse email	<code>demo@sillon.local</code>
Mot de passe	<code>demo</code>
Profil	Agent

Ce mot de passe est volontairement fixe et documenté, pour faciliter une session de démonstration ou de formation. Le paquet `sillon-tutoriel` ne doit **jamais** être installé sur un serveur de production — voir le guide d'installation administrateur, §7.2.

Le jeu de données

Trois tables sont déjà importées dans votre base personnelle :

- `communes_france` — 34 868 communes de France (métropole et outre-mer), avec pour chacune : son département, sa région, sa population, sa superficie, sa densité, son altitude et ses coordonnées.
- `regions_france` — les 18 régions françaises et le code INSEE de leur chef-lieu (préfecture de région), pour pratiquer les jointures.
- `contours_departements` — les frontières réelles des 104 départements français (métropole et outre-mer), sous forme de points ordonnés, pour la cartographie (partie 4).

Sources : jeu de données « Communes et villes de France », publié sur data.gouv.fr, construit à partir des données officielles de l'INSEE et de l'IGN ; contours départementaux issus de l'**IGN ADMIN EXPRESS**. Les deux sont diffusés sous **Licence Ouverte / Open Licence version 2.0** (Etalab).

Colonnes de `communes_france` :

Colonne	Type	Description
<code>code_insee</code>	Texte	Code INSEE de la commune
<code>nom_standard</code>	Texte	Nom de la commune
<code>dep_code</code>	Texte	Code du département
<code>dep_nom</code>	Texte	Nom du département
<code>reg_code</code>	Texte	Code de la région
<code>reg_nom</code>	Texte	Nom de la région
<code>population</code>	Entier	Population municipale
<code>superficie_km2</code>	Décimal	Superficie en km ²
<code>densite</code>	Décimal	Densité en habitants/km ²
<code>altitude_moyenne</code>	Entier	Altitude moyenne en mètres
<code>latitude_centre</code> / <code>longitude_centre</code>	Décimal	Coordonnées du centre de la commune

Colonnes de `regions_france` : `reg_code`, `reg_nom`, `chef_lieu_nom`, `chef_lieu_code_insee`.

Colonnes de `contours_departements` : `dep_code`, `dep_nom`, `groupe` (numéro d'anneau — un département avec des îles a plusieurs anneaux), `ordre` (position du point dans l'anneau), `longitude`, `latitude`.

Partie 1 — SQL progressif

Tous les exercices se travaillent dans l'onglet **Travaux**, en sélectionnant votre base personnelle. Les sept premiers exercices sont déjà présents dans votre **historique** (onglet Travaux) : vous pouvez les relire ou les réexécuter avant de passer aux suivants, qui restent à écrire vous-même.

Niveau 1 — Sélection et filtrage

1.1. Affichez toutes les communes du département du Rhône (code `69`).

```
SELECT * FROM communes_france WHERE dep_code = '69';
```

1.2. Affichez uniquement le nom et la population des communes de plus de 50 000 habitants.

```
SELECT nom_standard, population FROM communes_france WHERE population > 50000;
```

1.3. *À vous de jouer* : affichez les communes de moins de 100 habitants situées en Corse (code région 94).

▼ Corrigé

```
SELECT nom_standard, dep_nom, population
FROM communes_france
WHERE reg_code = '94' AND population < 100;
```

Niveau 2 — Tri et limitation

2.1. Les 10 communes les plus peuplées de Bretagne (code région 53).

```
SELECT nom_standard, population
FROM communes_france
WHERE reg_code = '53'
ORDER BY population DESC
LIMIT 10;
```

2.2. *À vous de jouer* : les 5 communes les plus hautes en altitude de France (attention aux valeurs manquantes : altitude_moyenne n'est pas toujours renseignée).

▼ Corrigé

```
SELECT nom_standard, dep_nom, altitude_moyenne
FROM communes_france
WHERE altitude_moyenne IS NOT NULL
ORDER BY altitude_moyenne DESC
LIMIT 5;
```

Niveau 3 — Agrégations

3.1. Nombre de communes et population totale du département de la Gironde (33).

```
SELECT COUNT(*) AS nb_communes, SUM(population) AS population_totale
FROM communes_france
WHERE dep_code = '33';
```

3.2. Population totale et nombre de communes par département, triés par population décroissante.

```
SELECT dep_nom, COUNT(*) AS nb_communes, SUM(population) AS population_totale
FROM communes_france
GROUP BY dep_nom
ORDER BY population_totale DESC;
```

3.3. *À vous de jouer* : la densité moyenne par région, arrondie à une décimale.

▼ Corrigé

```
SELECT reg_nom, ROUND(AVG(densite)::numeric, 1) AS densite_moyenne
FROM communes_france
GROUP BY reg_nom
ORDER BY densite_moyenne DESC;
```

Niveau 4 — Filtrer sur un agrégat (HAVING)

4.1. Les régions dont la densité moyenne dépasse 200 habitants/km².

```
SELECT reg_nom, ROUND(AVG(densite)::numeric, 1) AS densite_moyenne
FROM communes_france
GROUP BY reg_nom
HAVING AVG(densite) > 200
ORDER BY densite_moyenne DESC;
```

4.2. *À vous de jouer* : les départements comptant plus de 500 communes.

▼ Corrigé

```
SELECT dep_nom, COUNT(*) AS nb_communes
FROM communes_france
GROUP BY dep_nom
HAVING COUNT(*) > 500
ORDER BY nb_communes DESC;
```

Niveau 5 — Sous-requêtes

5.1. Les communes plus peuplées que la moyenne nationale.

```
SELECT nom_standard, dep_nom, population
FROM communes_france
WHERE population > (SELECT AVG(population) FROM communes_france)
ORDER BY population DESC;
```

5.2. *À vous de jouer* : pour chaque région, la commune la plus peuplée (indice : une sous-requête corrélée, ou une fonction de fenêtrage — voir niveau 7).

▼ **Corrigé (sous-requête corrélée)**

```
SELECT c.nom_standard, c.reg_nom, c.population
FROM communes_france c
WHERE c.population = (
    SELECT MAX(c2.population) FROM communes_france c2 WHERE c2.reg_nom =
    c.reg_nom
)
ORDER BY c.population DESC;
```

Niveau 6 — Jointures

6.1. Pour chaque région, la population de son chef-lieu (jointure avec `regions_france`).

```
SELECT c.nom_standard AS commune, c.population AS population_chef_lieu,
r.reg_nom
FROM communes_france c
JOIN regions_france r ON c.code_insee = r.chef_lieu_code_insee
ORDER BY c.population DESC;
```

6.2. *À vous de jouer* : la part de la population régionale que représente le chef-lieu (combinez la requête ci-dessus avec une agrégation de population par région).

▼ **Corrigé**

```
WITH population_regions AS (
    SELECT reg_nom, SUM(population) AS population_totale
    FROM communes_france
    GROUP BY reg_nom
)
SELECT
    c.nom_standard AS chef_lieu,
    c.population AS population_chef_lieu,
    pr.population_totale,
    ROUND(100.0 * c.population / pr.population_totale, 1) AS part_pourcent
FROM communes_france c
JOIN regions_france r ON c.code_insee = r.chef_lieu_code_insee
JOIN population_regions pr ON pr.reg_nom = r.reg_nom
ORDER BY part_pourcent DESC;
```

Niveau 7 — Fonctions de fenêtrage (avancé)

7.1. Le classement de chaque commune par population, au sein de son département (`RANK()` `OVER`).

```
SELECT
    nom_standard, dep_nom, population,
    RANK() OVER (PARTITION BY dep_code ORDER BY population DESC) AS
rang_departemental
FROM communes_france
ORDER BY dep_nom, rang_departemental;
```

7.2. *À vous de jouer* : ne garder que la commune la plus peuplée de chaque département (indice : enveloppez la requête précédente et filtrez sur `rang_departemental = 1`).

▼ Corrigé

```
SELECT * FROM (
    SELECT
        nom_standard, dep_nom, population,
        RANK() OVER (PARTITION BY dep_code ORDER BY population DESC) AS
rang_departemental
    FROM communes_france
) classement
WHERE rang_departemental = 1
ORDER BY population DESC;
```

Pour aller plus loin : le délai maximal d'une requête est limité (§11) ; une requête plus longue peut être basculée en tâche de fond (bouton « Exécuter en tâche de fond »), avec notification par mail à la fin — utile si vous étendez ces exercices à des calculs plus lourds.

Partie 2 — Python avancé

L'onglet **Scripts** permet de déposer un fichier `.py` ou `.R`, exécuté dans un environnement isolé avec un accès direct à votre base (variable d'environnement `SILLON_DSN`) et un répertoire de sortie pour vos résultats (`SILLON_RESULTATS`). Bibliothèques Python disponibles : `pandas`, `numpy`, `matplotlib`, `psycopg2`, `openpyxl` — consultez la liste à jour dans l'onglet avant de coder. Pas de librairie géospatiale (geopandas, shapely) ni de graphiques interactifs (plotly, bokeh) : tout export est une image statique ou un fichier.

Squelette commun à tous les scripts de cette partie :

```
import os
import psycopg2
import pandas as pd

dsn = os.environ["SILLON_DSN"]
resultats = os.environ["SILLON_RESULTATS"]

connexion = psycopg2.connect(dsn)
communes = pd.read_sql("SELECT * FROM communes_france", connexion)
connexion.close()

# ... votre analyse, vos graphiques, vos exports ...
```

2.1 Panorama des graphiques (matplotlib)

`matplotlib.use("Agg")` est **obligatoire** en tout début de script (avant `import matplotlib.pyplot`) : aucun affichage interactif n'est possible dans le conteneur d'exécution.

Graphique	Fonction	Usage typique
Courbe	<code>ax.plot(x, y)</code>	évolution le long d'une variable continue
Barres	<code>ax.bar(x, y)</code> / <code>ax.barh(x, y)</code>	comparaison de catégories (verticale/horizontale)
Nuage de points	<code>ax.scatter(x, y)</code>	relation entre deux variables
Histogramme	<code>ax.hist(valeurs, bins=...)</code>	distribution d'une variable
Camembert	<code>ax.pie(valeurs, labels=...)</code>	répartition en proportions (à réserver à moins de 6 catégories, sinon illisible)
Boîte à moustaches	<code>ax.boxplot([groupe1, groupe2, ...])</code>	comparer la dispersion entre groupes
Carte de chaleur	<code>ax.imshow(matrice, cmap=...)</code>	matrice de valeurs (corrélations, tableaux croisés)
Plusieurs graphiques	<code>fig, axes = plt.subplots(2, 2)</code>	tableau de bord en une seule image

Exemple : une carte de chaleur des corrélations entre les variables numériques de `communes_france`.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

colonnes_numeriques = ["population", "superficie_km2", "densite",
                        "altitude_moyenne"]
correlations = communes[colonnes_numeriques].corr()

figure, axe = plt.subplots(figsize=(6, 5))
image = axe.imshow(correlations, cmap="coolwarm", vmin=-1, vmax=1)
axe.set_xticks(range(len(colonnes_numeriques)), colonnes_numeriques,
               rotation=45, ha="right")
axe.set_yticks(range(len(colonnes_numeriques)), colonnes_numeriques)
plt.colorbar(image, ax=axe, label="Corrélation")
figure.savefig(os.path.join(resultats, "correlations.png"), bbox_inches="tight")
```

Le script d'exemple `analyse_demographie.py` (déjà déposé et exécuté, résultat dans l'onglet **Suivi**) combine barres horizontales, barres verticales et histogramme en échelle logarithmique sur les 34 868 communes.

2.2 Tableaux et export Excel

Un tableau croisé avec `pandas.pivot_table` :

```
tableau = communes.pivot_table(
    index="reg_nom", values="population", aggfunc=["count", "sum", "mean"]
)
```

Export vers un vrai classeur Excel avec `openpyxl` (installé, pas seulement `pandas.to_csv`) : plusieurs onglets, mise en forme des cellules, et même des **graphiques natifs Excel** (pas une image collée) :


```

from openpyxl import Workbook
from openpyxl.chart import BarChart, Reference
from openpyxl.styles import Font

classeur = Workbook()
feuille = classeur.active
feuille.title = "Population par région"

par_region = communes.groupby("reg_nom")
["population"].sum().sort_values(ascending=False)
feuille.append(["Région", "Population"])
feuille["A1"].font = feuille["B1"].font = Font(bold=True)
for region, population in par_region.items():
    feuille.append([region, int(population)])

graphique = BarChart()
graphique.title = "Population par région"
donnees = Reference(feuille, min_col=2, min_row=1, max_row=feuille.max_row)
categories = Reference(feuille, min_col=1, min_row=2, max_row=feuille.max_row)
graphique.add_data(donnees, titles_from_data=True)
graphique.set_categories(categories)
feuille.add_chart(graphique, "D2")

classeur.save(os.path.join(resultats, "rapport.xlsx"))

```

2.3 Rapports PDF multi-pages

`matplotlib.backends.backend_pdf.PdfPages` assemble plusieurs graphiques dans un seul PDF, une page par figure — pratique pour un rapport complet en une seule pièce jointe :

```

from matplotlib.backends.backend_pdf import PdfPages

with PdfPages(os.path.join(resultats, "rapport.pdf")) as pdf:
    figure1, ax1 = plt.subplots()
    ax1.hist(communes["densite"].dropna(), bins=50)
    ax1.set_title("Distribution de la densité")
    pdf.savefig(figure1)
    plt.close(figure1)

    figure2, axe2 = plt.subplots()
    axe2.bar(par_region.index, par_region.values)
    axe2.set_title("Population par région")
    plt.xticks(rotation=60, ha="right")
    pdf.savefig(figure2, bbox_inches="tight")
    plt.close(figure2)

```

2.4 Cartographie

Pas de `geopandas` / `shapely` dans l'image d'exécution : la table `contours_departements` aplatit volontairement chaque polygone en points ordonnés (`dep_code` , `groupe` , `ordre` , `longitude` , `latitude`) plutôt qu'un format géospatial — `matplotlib.patches.Polygon` sait tracer un polygone à partir d'une simple liste de coordonnées, sans dépendance supplémentaire.

Le script d'exemple `carte_departements.py` (déjà déposé et exécuté, résultat dans l'onglet [Suivi](#)) trace une carte choroplèthe de la population par département :

```
from collections import defaultdict
from matplotlib.patches import Polygon
from matplotlib.collections import PatchCollection

contours = pd.read_sql(
    "SELECT dep_code, groupe, ordre, longitude, latitude FROM
    contours_departements ORDER BY dep_code, groupe, ordre",
    connexion,
)
polygones = defaultdict(list)
for ligne in contours.itertuples():
    polygones[(ligne.dep_code, ligne.groupe)].append((ligne.longitude,
    ligne.latitude))

patches = [Polygon(points, closed=True) for points in polygones.values()]
# ... couleur de chaque polygone selon la valeur de son département (cf. script
complet) ...
collection = PatchCollection(patches, cmap="Blues")

figure, axe = plt.subplots(figsize=(8, 8))
axe.add_collection(collection)
axe.autoscale_view()
axe.set_aspect("equal")
axe.axis("off")
figure.savefig(os.path.join(resultats, "carte.png"), bbox_inches="tight")
```

Exercices

2.1. Reproduisez la carte de `carte_departements.py` , mais coloriée par **densité moyenne** du département plutôt que par population totale.

2.2. Exportez un fichier Excel avec un onglet par région, chacun listant les communes de la région triées par population, en utilisant `openpyxl` (indice : une feuille par valeur unique de `reg_nom` , via une boucle).

2.3. Avancé : assemblez en un seul PDF (`PdfPages`) trois graphiques de votre choix parmi ceux du panorama (2.1).

2.4. Avancé : calculez, pour chaque région, le coefficient de corrélation entre altitude moyenne et densité (`DataFrame.corr()` après un `groupby`). Les régions de montagne sont-elles significativement moins denses ?

Partie 3 — R avancé

Même contrat d'exécution qu'en Python (`SILLON_DSN`, `SILLON_RESULTATS`). Librairies R disponibles : `tidyverse` (dont `dplyr`, `ggplot2`, `tidyr`, `readr`), `DBI`, `RPostgreSQL`. **Pas de `jsonlite`, `sf` ni `openxlsx`** dans l'image d'exécution : contrairement à Python, R ne peut donc pas écrire de fichier Excel ici — seulement du CSV (`write.csv`) et des graphiques.

Point d'attention : `RPostgreSQL::dbConnect()` n'accepte pas directement la chaîne de connexion `SILLON_DSN` (format `clé=valeur` de `libpq`) — il faut l'analyser vous-même :

```
library(DBI)
library(RPostgreSQL)
library(dplyr)
library(ggplot2)

analyser_dsn <- function(dsn) {
  valeurs <- list()
  for (paire in strsplit(trimws(dsn), "\\s+")[1]) {
    cle_valeur <- strsplit(paire, "=", fixed = TRUE)[1]
    valeurs[[cle_valeur[1]]] <- cle_valeur[2]
  }
  valeurs
}

parametres <- analyser_dsn(Sys.getenv("SILLON_DSN"))
resultats <- Sys.getenv("SILLON_RESULTATS")

connexion <- dbConnect(
  PostgreSQL(),
  host = parametres$host, port = as.integer(parametres$port),
  dbname = parametres$dbname, user = parametres$user, password =
parametres$password
)
communes <- dbGetQuery(connexion, "SELECT * FROM communes_france")
```

3.1 Panorama des graphiques (ggplot2)

Toujours la même grammaire : `ggplot(donnees, aes(...)) + geom_...() + ...`.

Graphique	Géométrie	Usage typique
Courbe	<code>geom_line()</code>	évolution le long d'une variable continue
Barres	<code>geom_col()</code> (valeurs déjà agrégées) / <code>geom_bar()</code> (comptage)	comparaison de catégories
Nuage de points	<code>geom_point()</code>	relation entre deux variables
Histogramme	<code>geom_histogram()</code>	distribution d'une variable
Boîte à moustaches	<code>geom_boxplot()</code>	comparer la dispersion entre groupes
Plusieurs graphiques	<code>facet_wrap(~ variable)</code>	un panneau par catégorie, échelle commune

```
graphique <- ggplot(communes, aes(x = densite)) +
  geom_histogram(bins = 50, fill = "#000091") +
  scale_x_log10() +
  facet_wrap(~ reg_nom) + # un histogramme par région
  theme_minimal()
ggsave(file.path(resultats, "densite_par_region.png"), graphique, width = 10,
height = 8)
```

Le script d'exemple `analyse_densite.R` (déjà exécuté, résultat dans l'onglet [Suivi](#)) trace la densité moyenne des 15 départements les plus denses et la relation altitude/densité.

3.2 Tableaux

```
tableau <- communes %>%
  group_by(reg_nom) %>%
  summarise(nb_communes = n(), population_totale = sum(population),
densite_moyenne = mean(densite, na.rm = TRUE)) %>%
  arrange(desc(population_totale))

write.csv(tableau, file.path(resultats, "tableau_regions.csv"), row.names =
FALSE)
```

3.3 Rapports PDF multi-pages

Le périphérique graphique `pdf()` de R (toujours disponible, aucun package requis) écrit une nouvelle page à chaque graphique tracé entre son ouverture et `dev.off()` :

```
pdf(file.path(resultats, "rapport.pdf"), width = 8, height = 6)

hist(communes$densite, breaks = 50, main = "Distribution de la densité")

par_region <- communes %>% group_by(reg_nom) %>% summarise(population_totale =
sum(population))
barplot(par_region$population_totale, names.arg = par_region$reg_nom, las = 2,
        main = "Population par région")

dev.off()
```

3.4 Cartographie

Sans `sf` ni `maps` dans l'image d'exécution : `geom_polygon()` de `ggplot2` sait tracer un polygone directement à partir d'un data frame de points ordonnés (colonnes `x`, `y`, `group`) — le style « fortifié » classique de `ggplot2`, sans dépendance géospatiale.

Le script d'exemple `carte_departements.R` (déjà déposé et exécuté, résultat dans l'onglet [Suivi](#)) trace la même carte choroplèthe qu'en Python :

```
contours <- dbGetQuery(
  connexion,
  "SELECT dep_code, groupe, ordre, longitude, latitude FROM
contours_departements ORDER BY dep_code, groupe, ordre"
)
population_par_dep <- dbGetQuery(
  connexion, "SELECT dep_code, SUM(population) AS population_totale FROM
communes_france GROUP BY dep_code"
)

contours <- contours %>%
  mutate(id_polygone = paste(dep_code, groupe, sep = "_")) %>%
  left_join(population_par_dep, by = "dep_code")

carte <- ggplot(contours, aes(x = longitude, y = latitude, group = id_polygone,
fill = population_totale)) +
  geom_polygon(color = "white", linewidth = 0.1) +
  coord_fixed() +
  scale_fill_gradient(low = "#e8edfb", high = "#000091") +
  theme_void()
ggsave(file.path(resultats, "carte.png"), carte, width = 8, height = 8)
```

Exercices

3.1. Reproduisez la carte de `carte_departements.R`, mais coloriée par densité moyenne plutôt que par population totale (indice : comparez à l'exercice Python 2.1).

3.2. Avec `facet_wrap`, tracez un histogramme de la population des communes, un panneau par région.

3.3. Avancé : à l'aide de `regions_france`, faites une jointure (`inner_join`) entre les deux tables pour comparer, région par région, la population du chef-lieu à la population totale de la région — comme l'exercice SQL 6.2, mais en R.

3.4. Avancé : assemblez trois graphiques dans un seul rapport PDF avec `pdf()` / `dev.off()`.

Pour continuer

- Les onglets **Bases**, **Import** et **Suivi** couvrent le reste des fonctionnalités de SILLON : import de vos propres fichiers CSV, partage de base avec un autre compte, suivi et annulation des traitements en cours.
- Le **Guide d'installation administrateur** (accessible depuis le bouton « À propos ») détaille la configuration du serveur, les quotas et les procédures de désinstallation.
- Ce compte de démonstration peut être réinitialisé à tout moment : `sudo apt-get purge sillon-tutoriel && sudo apt-get install sillon-tutoriel` (voir le guide d'installation administrateur, §7.2 et §8).